Programming Techniques

Week 03: Data Structures & Algorithmic Thinking

MASTER OF INFORMATION TECHNOLOGY
SABARAGAMUWA UNIVERSITY OF SRI LANKA

Learning Objectives

Structures

Understand how data is organized in memory.

Lists

Master Python lists for collection management.

Logic

Apply algorithmic thinking to solve problems.

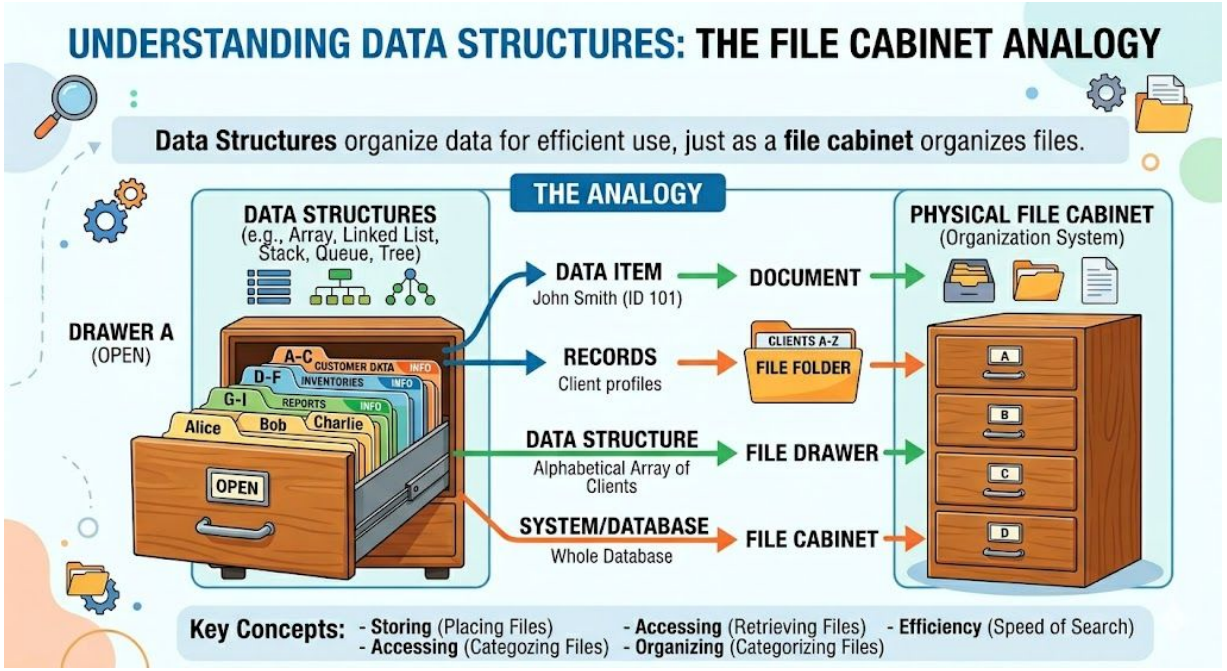
Systems

Build reusable functions for data processing.

What is a Data Structure?

Definition: A specialized format for organizing, processing, retrieving, and storing data.

Real-world Analogy: A Filing Cabinet. We don't just throw papers in a pile; we use folders and labels for quick retrieval.



https://www.youtube.com/watch?v=IiVadPS3AhY

Types of Data Structures

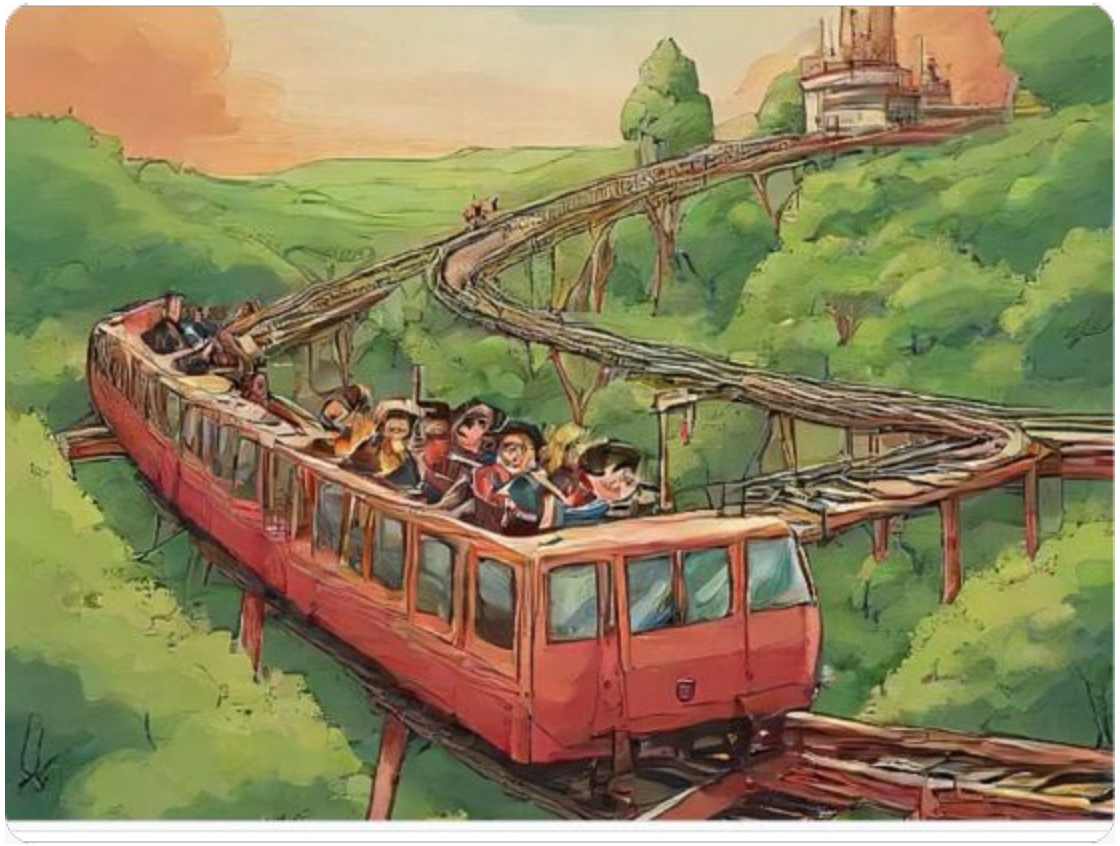
Category	Examples	Description
Primitive	int, float, string, bool	Store a single value at a time.
Non-Primitive	list, dictionary, tuple, set	Store collections of values or complex objects.

Introduction to Lists

A **List** is a collection of items in a specific order.

Analogy: A Cargo Train. Each container holds a piece of data, and they are linked in a sequence.

```
numbers  = [10, 20, 30]
```



List Characteristics

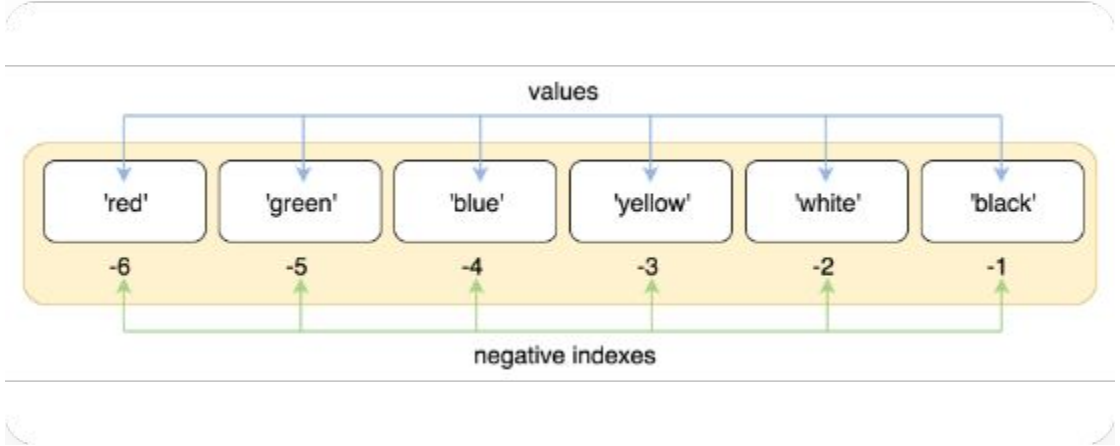
- **Ordered:** Elements maintain their defined position.
- **Mutable:** You can change, add, or remove items after creation.
- **Allows Duplicates:** Can hold multiple instances of the same value.
- **Dynamic:** Can hold different data types in the same list.

The Indexing Concept

Python lists use **Zero-based Indexing**.

To access an item, we use its position number inside square brackets [].

Indices: 0, 1, 2, 3...



```
# Output: [10, 20, 100]
```

Accessing & Updating

We use the index to read a specific value or overwrite it.

Syntax: `list_name[index]`

```
# Accessing
print(numbers[0])  # Output: 10

# Updating
numbers[2] = 100
print(numbers)  # Output: [10, 20, 100]
```

Traversing a List

Traversal means visiting every item in a list once.

The `for...in` loop is the most elegant way to traverse in Python.

```
for num in numbers:
    print(num)

# Logic: Each item is assigned
# to 'num' temporarily.
```

Index-based Traversal

Sometimes you need the **position (index)** of the item while looping.

We use `range()` combined with `len()`.

```
for i in range(len(numbers)):
    print("Index:", i)
    print("Value:", numbers[i])
```

What is an Algorithm?

Steps

A finite sequence of well-defined instructions.

Solution

Designed to solve a specific class of problems.

Example

A recipe for making a sandwich is an algorithm.

Problem-Solving Framework

Understand	Plan	Logic	Code
Identify inputs & outputs.	Sketch the steps (Pseudocode).	Define the control flow (Loops/Ifs).	Translate logic into Python.

Algorithm: Sum of List

Goal: Add all numbers in a list.

Strategy: Initialize an accumulator variable to zero, then add each item during a loop.

```
total = 0
for num in numbers:
    total += num

print("Sum is:", total)
```

"Max is:"

Algorithm: Maximum Value

Strategy: Assume the first item is the largest. Compare it with every other item.

If we find a bigger item, update our `max_val`.

```
max_val = numbers[0]
for num in numbers:
    if num > max_val:
        max_val = num

print("Max is:", max_val)
```

Algorithm: Counting Evens

Strategy: Use a counter variable. Add 1 only if the condition (modulo) is met.

This demonstrates **Decision Making** inside a **Loop**.

```
count = 0
for num in numbers:
    if num % 2 == 0:
        count += 1

print("Even count:", count)
```

Introduction to Efficiency

O(n)

TIME COMPLEXITY

Efficiency measures how **resources** change as your **data size (n)** grows.

Looping through a list once is "Linear Time." If the list doubles, the time doubles.

Functions with Lists

Pass a list as an **Argument** to a function to make your logic reusable.

This encapsulates your algorithm inside a black box.

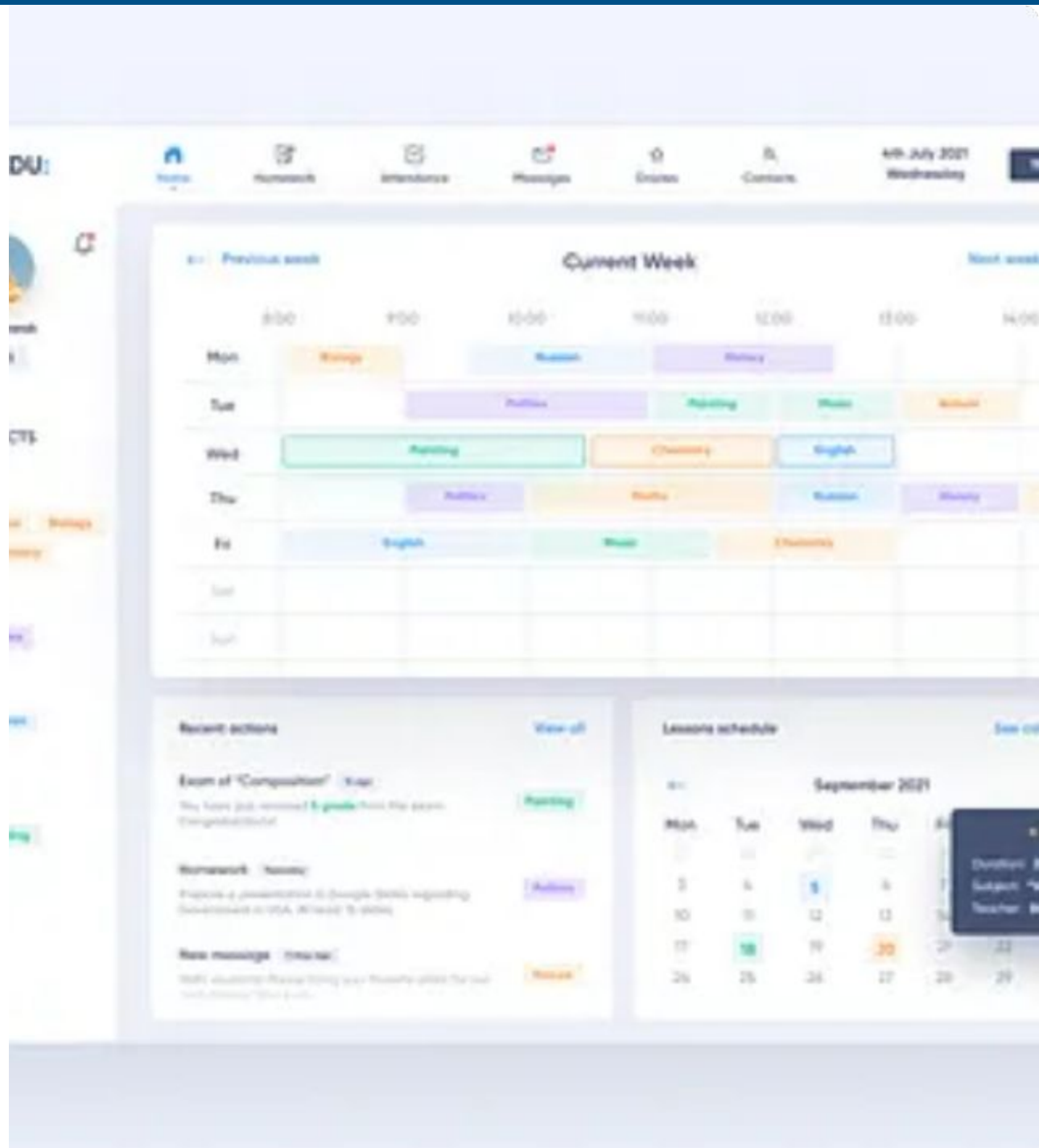
```
def find_average(numbers):
    total = sum(numbers)
    count = len(numbers)
    return total / count

avg = find_average([80, 90, 100])
```

Student Marks System

Use Case: Processing class performance.

- **Input:** List of marks.
- **Processing:** Calc Avg & Max.
- **Output:** Grade Statistics.



What is Pseudocode?

- It is "fake code" for human reading.
- Not strictly Python, but follows the same logic.

```
1. START
2. SET TOTAL to 0
3. FOR each MARK in LIST
4.   ADD MARK to TOTAL
5. PRINT TOTAL / LENGTH
6. END
```

Exercise: Basic Lists

Create

Create a list of 5 colors.

Display

Print the 2nd and 4th color.

Measure

Print the total number of colors.

Exercise: Processing Data

Sum

Find the sum of all elements.

Maximum

Identify the largest number.

Count

Count how many numbers are even.

Exercise: Transformation

Reverse

Print the list in reverse order.

Runner-up

Find the second largest number.

Unique

Remove all duplicate values.

Group Activity

Student Performance System

Activity Instructions

1. **Logic:** Explain your flowchart.
2. **Code:** Show your list-processing function.
3. **Results:** Discuss the stats calculated.

Goal

Analyze a list of 10 student marks and output: Pass count, Avg mark, and Top mark.

Common Mistakes

- **IndexError:** Trying to access list[5] when length is only 5 (valid indices are 0-4).
- **Empty Lists:** Forgetting to check if a list is empty before finding Max.
- **Off-by-one:** Miscounting range limits in loops.

Summary

- **Lists:** Fundamental storage for data collections.
- **Algorithms:** Step-by-step logic for processing that data.
- **Functions:** Encapsulate algorithms for modular systems.

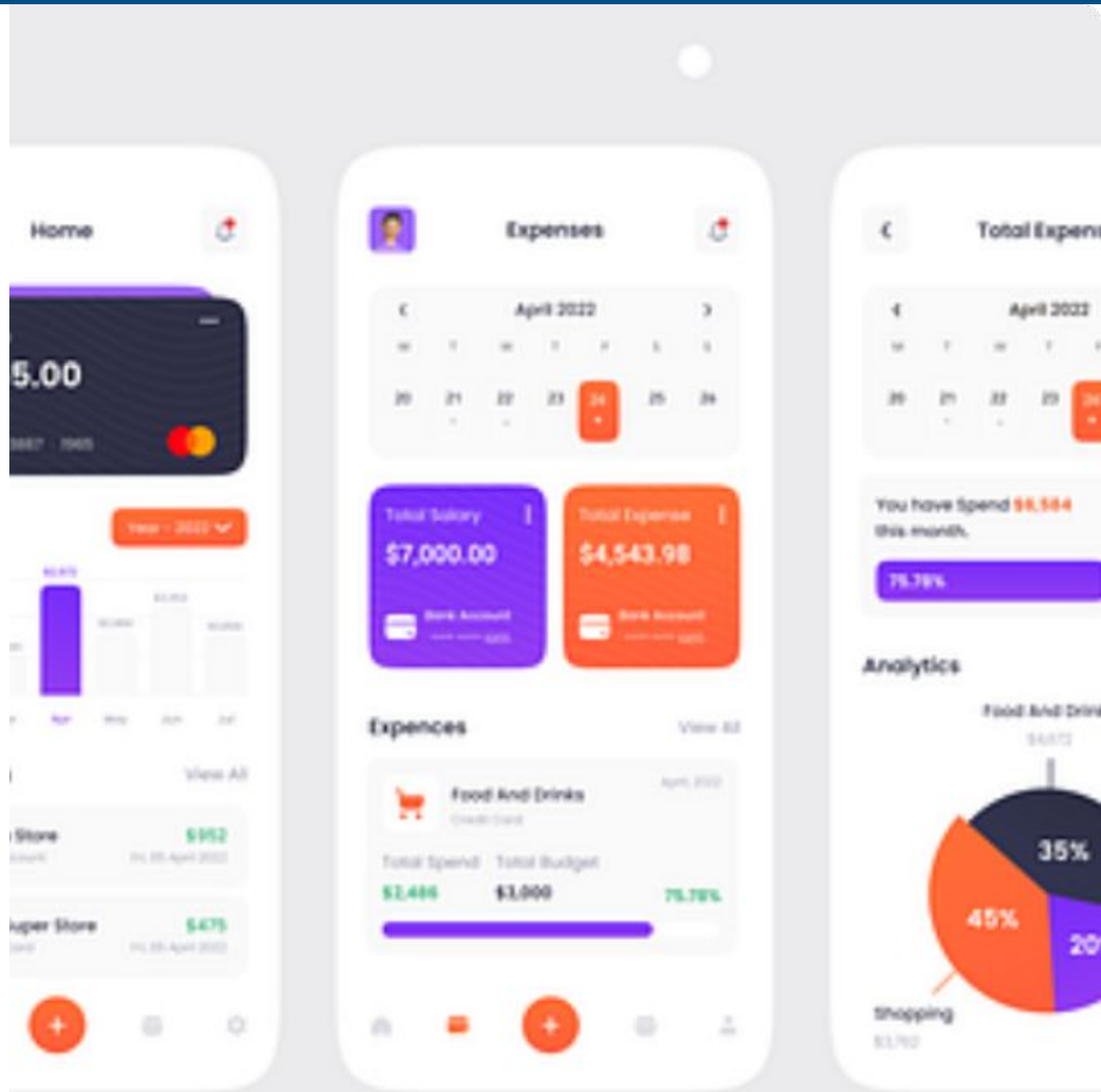
Homework: Expense Analyzer

Requirement: Store your daily expenses in a list.

Calculate total spending.

Find the single highest expense.

Create a function is_over_budget(total, limit).



Thank You!

Any Questions?

Week 3: Data Structures & Algorithmic Thinking
Next Week: Dictionaries & Complexity Analysis

Image Sources

https://i.ytimg.com/vi/IiVadPS3AhY/hq720.jpg?sqp=-oaymwE7CK4FEIIDSFryq4qpAy0IARUAAAAAGAElAADIQj0AgKJD8AEB-AH-BYAC4AOKAgwIABABGGUgYShRMA8=&rs=AOn4CLBKSHDW3rE6m08KCocPYwzW_qF0Dw

Source: www.youtube.com

https://media2.dev.to/dynamic/image/width=800%2Cheight=%2Cfit=scale-down%2Cgravity=auto%2Cformat=auto/https%3A%2F%2Fdev-to-uploads.s3.amazonaws.com%2Fuploads%2Farticles%2Fqnswqky5dyq74pcvcjc6.png

Source: dev.to

https://railsware.com/blog/wp-content/uploads/2018/10/negative-indexes.png

Source: railsware.com

https://cdn.dribbble.com/userupload/32916209/file/original-5af43d633ce4998ac1b3cf4867cff131.png?format=webp&resize=400x300&vertical=center

Source: dribbble.com

https://cdn.dribbble.com/userupload/37365074/file/original-e10b367e6473ae749eab12c592cc5336.png?resize=400x0

Source: dribbble.com